

深度求索web前端高级工程师-AIGC

一面

1. 请简述一下 Server-Sent Events (SSE) 的工作原理，以及它与 WebSocket 的区别。

难度：★★

考点：流式传输、网络协议

答案要点：

SSE 是一种基于 HTTP 协议的服务器向浏览器推送实时数据的机制，常用于 AI 聊天的流式响应。

- 协议基础：SSE 基于标准 HTTP 协议，使用 text/event-stream 格式，是单向通信（服务端到客户端）。
- 连接特性：支持断线重连（浏览器自动处理），相比 WebSocket 更轻量，且能穿透大多数防火墙。
- 数据格式：传输的是纯文本，通过特定的字段（如 data, event, id）进行消息分割。
- 适用场景：非常适合 LLM 这种只需要服务端持续输出结果，而客户端较少发送高频指令的场景。

常见坑：

- 忽略了浏览器对同一域名的 HTTP/1.1 最大连接数限制（通常为 6 个）。
- 忘记在服务端设置 Cache-Control: no-cache。

追问：

- 如果要在 SSE 中实现类似 WebSocket 的双向交互，你会怎么做？

2. 在处理 AI 返回的 Markdown 文本流时，如何保证前端渲染的性能和稳定性？

难度：★★

考点：DOM 渲染、流式解析

答案要点：

处理流式 Markdown 需要平衡渲染频率和用户感知的流畅度，避免因频繁操作 DOM 导致页面卡顿。

- 增量解析：使用支持流式解析的 Markdown 库（如 markdown-it 的插件或自定义逻辑），只对新到达的片段进行处理。
- 渲染节流：不要每收到一个 token 就触发一次 React/Vue 的状态更新，可以利用 requestAnimationFrame 进行批量渲染。
- 闭合标签处理：在流式传输过程中，Markdown 标签（如代码块、加粗）可能是不完整的，需要有容错机制防止样式崩坏。

- 自动滚动控制：实现类似 ChatGPT 的自动探底功能，但要判断用户是否正在手动向上滚动以停止自动跟随。

常见坑：

- 每次更新都全量重新渲染整个对话框，导致长对话时输入框输入延迟。
- 代码高亮库（如 Prism.js）在流式更新时反复执行导致的 CPU 占用过高。

追问：

- 如何处理流式输出中途突然中断导致的代码块未闭合问题？

3. 说一下 Fetch API 中的 ReadableStream 是如何使用的，如何手动取消一个请求？

难度：★★

考点：Fetch API、流处理、AbortController

答案要点：

Fetch API 的 `response.body` 是一个 `ReadableStream`，允许我们在数据全部到达前逐块读取。

- 读取流程：通过 `getReader()` 获取读取器，在循环中调用 `read()` 方法获取 `value` (`Uint8Array`) 和 `done` 状态。
- 数据转换：使用 `TextDecoder` 将字节流转换为字符串，以便进行后续的业务逻辑处理。
- 请求取消：通过实例化 `AbortController`，将其 `signal` 传递给 `fetch` 的配置项，调用 `abort()` 方法即可中断连接。
- 资源释放：在读取结束或发生错误时，调用 `reader.releaseLock()` 释放读取器。

常见坑：

- 忘记处理 `TextDecoder` 在字符被截断时的乱码问题（应使用 `stream: true` 参数）。
- 以为取消请求后服务端会立刻感知并停止生成（实际上取决于后端实现）。

追问：

- 如果一个页面有多个并发的 AI 请求，如何精准取消其中某一个？

4. 谈谈 TypeScript 中的泛型（Generics），并写一个用于包装 AI 接口返回值的通用类型。

难度：★

考点：TS 基础、类型安全

答案要点：

泛型是指在定义函数、接口或类时不预先指定具体类型，而在使用时才指定类型的一种特性。

- 核心作用：提高代码的可复用性和类型安全性，避免过度使用 `any`。
- 类型约束：可以使用 `extends` 关键字对泛型进行约束，确保传入的类型符合特定结构。
- 默认参数：可以为泛型指定默认类型，简化调用。

```
代码块
1 interface ApiResponse<T = any> {
2     code: number;
3     data: T;
4     message: string;
5     usage?: {
6         prompt_tokens: number;
7         completion_tokens: number;
8     };
9 }
```

常见坑：

- 泛型嵌套过深导致类型推导失效或代码可读性极差。
- 在不需要泛型的地方强行使用，增加复杂度。

追问：

- 什么是泛型实例化中的“逆变”与“协变”？

5. 如何实现一个支持自动换行且高度随内容自适应的聊天输入框？

难度：★

考点：CSS 布局、DOM 操作

答案要点：

实现高度自适应输入框通常有两种主流方案：利用 textarea 的 scrollHeight 或使用 contenteditable 元素。

- Textarea 方案：监听 input 事件，先将 height 设为 auto，再赋值为 scrollHeight，需注意 padding 和 border 的计算。
- Contenteditable 方案：原生支持高度自适应，但需要处理纯文本粘贴、光标位置控制等复杂的交互细节。
- 现代 CSS 方案：使用 field-sizing: content（实验性属性）或者利用 grid 布局的 fr 单位配合隐形镜像元素。
- 交互细节：处理 Shift + Enter 换行与 Enter 发送的逻辑冲突。

常见坑：

- 忽略了最大高度（max-height）的限制，导致输入框无限拉长遮盖对话区域。
- 粘贴长文本时没有处理滚动条抖动。

追问：

- 如何在 contenteditable 元素中实现 @ 提及功能并保持光标位置正确？

6. 写一个简单的防抖（Debounce）函数，并说明它在 AI 搜索建议场景中的作用。

难度：★

考点：手写代码、性能优化

答案要点：

防抖是指在事件被触发 n 秒后再执行回调，如果在这 n 秒内又被触发，则重新计时。

- 实现要点：闭包存储 timer 变量，返回一个新函数，内部使用 clearTimeout 和 setTimeout。
- 业务价值：在 AI 搜索建议中，防止用户每输入一个字符就请求一次昂贵的 LLM 接口，节省 Token 和服务器压力。
- 立即执行：高级版本通常支持 leading 参数，决定是否在第一次触发时立即执行。

代码块

```
1 function debounce(fn, delay) {  
2   let timer = null;  
3   return function(...args) {  
4     if (timer) clearTimeout(timer);  
5     timer = setTimeout(() => {  
6       fn.apply(this, args);  
7     }, delay);  
8   };  
9 }
```

常见坑：

- 忘记绑定 this 指向。
- 在 React 组件中直接定义防抖函数，导致每次 render 都生成新的函数使防抖失效（应使用 useMemo 或 useCallback）。

追问：

- 节流（Throttle）和防抖的区别是什么？在 AI 聊天自动滚动场景中该用哪个？

二面（深入原理与项目）

1. React 的 Fiber 架构是如何提升复杂 AI 应用的交互流畅度的？

难度：★★★

考点：React 原理、调度算法

答案要点：

Fiber 架构将同步不可中断的更新变成了异步可中断的更新，解决了主线程长时间被占用导致的掉帧问题。

- 时间分片：Fiber 将渲染工作拆分成多个小单元，在浏览器空闲时间执行，避免阻塞 AI 响应流的实时渲染。

- 优先级调度：AI 聊天中的用户输入（Input）具有高优先级，而历史消息的后台加载或非视口内的渲染可以设为低优先级。
- 双缓存技术：在内存中构建新的 Fiber 树，完成后一次性提交到真实 DOM，减少中间状态的闪烁。
- 协调算法：通过 diff 算法最小化 DOM 操作，这在频繁更新的流式输出场景下至关重要。

常见坑：

- 误以为 Fiber 能让代码执行变快，实际上它只是让页面“看起来”更流畅。
- 在 render 阶段执行了带有副作用的操作。

追问：

- 如何在 React 中利用 useTransition 来优化 AI 生成内容的展示？

2. 在 AI 聊天应用中，如果对话历史非常长（上千条），你会采取哪些前端优化策略？

难度：★★★★

考点：长列表优化、内存管理

答案要点：

长对话会导致 DOM 节点过多和内存占用过高，必须通过虚拟滚动和数据分段加载来解决。

- 虚拟列表（Virtual List）：只渲染当前视口可见的对话卡片，通过 padding 或 transform 模拟滚动条高度。
- 动态高度计算：AI 回复长度不一，需要实时测量或预估 DOM 高度，并缓存已渲染节点的位置信息。
- 消息分页/懒加载：初始只加载最近的 20 条，向上滚动时再按需拉取历史记录。
- 内存清理：对于不可见区域的复杂组件（如公式渲染、代码编辑器），可以卸载其实例以释放内存。

常见坑：

- 虚拟列表滚动过快时的白屏问题。
- 聊天记录中包含图片或 iframe 时，高度塌陷导致的滚动位置跳变。

追问：

- 如何在不使用第三方库的情况下实现一个简单的虚拟列表？

3. 谈谈 Prompt Injection（提示词注入）在前端层面的防范措施。

难度：★★

考点：安全、AI 交互

答案要点：

虽然提示词注入主要在后端防御，但前端作为交互入口，也需要承担部分校验和隔离工作。

- 输入过滤：在前端对用户输入进行敏感词扫描或格式校验，拦截明显的恶意指令（如 "Ignore previous instructions"）。
- 角色隔离：在 UI 上明确区分“系统指令”、“用户输入”和“AI 回复”，防止用户伪造系统 UI 诱导他人。
- 输出转义：对 AI 返回的内容进行严格的 XSS 过滤，特别是当 AI 试图输出 HTML 或 Script 标签时。
- Content Security Policy (CSP)：配置严格的 CSP 策略，限制 AI 生成内容中潜在的恶意脚本执行或非法外链请求。

常见坑：

- 盲目信任 AI 返回的 Markdown 里的链接，可能包含 javascript: 伪协议。
- 允许 AI 直接操作 DOM 或执行 eval。

追问：

- 如果 AI 需要输出一个预览网页的功能，你如何安全地在前端展示它？

4. 如何设计一个通用的 AI 组件库，使其能适配不同的模型协议（如 OpenAI, Anthropic, 自研模型）？

难度：★★★

考点：架构设计、抽象能力

答案要点：

设计核心在于解耦 UI 渲染层与底层通信协议层，通过适配器模式实现多模型支持。

- 统一数据模型：定义一套标准的消息格式（如 Message 接口），包含 role, content, type, status 等字段。
- 适配器层 (Adapter)：针对不同模型的 API 编写适配器，将特定的流式响应格式转换为前端通用的 Observable 或 AsyncIterable。
- 状态机管理：使用 XState 或简单的状态机管理对话状态（Init, Sending, Streaming, Error, Success）。
- 插件化能力：支持通过插件扩展 AI 的能力，如工具调用（Tool Use）的 UI 展示、多模态附件处理等。

常见坑：

- UI 组件与特定的 SDK（如 OpenAI SDK）强耦合，导致切换模型时需要重构大量代码。
- 忽略了不同模型在流式输出结束标识上的差异。

追问：

- 你的组件库如何处理多轮对话中的 Context 长度限制提示？

5. 详细说明如何实现 AI 聊天中的“停止生成”功能，包括前端和后端的协作逻辑。

难度：★★

考点：请求控制、流中断

答案要点：

停止生成不仅是前端 UI 的停止，更重要的是及时中断网络连接以节省计算资源。

- 前端中断：利用 `AbortController.abort()` 立即断开当前的 HTTP/SSE 连接。
- 状态同步：前端调用 `abort` 后，需立即更新 UI 状态为“已停止”，并处理可能已经到达但未渲染的残余数据。
- 后端感知：后端需要监听 HTTP 连接的断开事件（如 Node.js 的 `req.on('close')`），并向上游模型 API 发送取消指令。
- 数据库一致性：如果对话需要入库，前端中断后需要通知后端标记该条消息为“已截断”而非“已完成”。

常见坑：

- 前端断开了连接，但后端仍在持续调用模型 API 计费。
- 停止后立刻发起新请求，导致旧请求的残余数据干扰新对话。

追问：

- 在高并发场景下，如何确保“停止”指令的实时性？

6. 你们在项目中是如何处理 AI 响应过慢（Latency）导致的体验问题的？

难度：★★

考点：用户体验、性能优化

答案要点：

AI 响应慢是行业通病，前端可以通过感知优化和技术手段缓解用户的焦虑感。

- 预加载与预执行：在用户输入时进行意图预测，提前建立连接或预取相关上下文。
- 渐进式渲染：利用流式输出，第一时间展示首个 token（TTFT 优化），并配合打字机动画效果。
- 占位与反馈：在等待首个字符期间，提供多样化的 Loading 状态或随机的“思考中”提示词。
- 边缘计算：将部分逻辑（如 Prompt 拼接、简单的格式化）放在 Edge Runtime 执行，减少往返时延。

常见坑：

- 打字机动画速度固定，导致动画跟不上流式传输的速度或显得太慢。
- 在等待期间禁用整个输入框，导致用户无法纠错或补充信息。

追问：

- 如何量化 AI 产品的首屏渲染时间（TTFT）？

7. 实现一个函数，将 AI 返回的非标准 JSON 字符串流实时解析为对象（Partial JSON Parsing）。

难度：★★★

考点：算法、流式解析

答案要点：

AI 在流式输出 JSON 时，中间状态是非法的 JSON 字符串，需要特殊处理才能实现“边出边看”。

- 递归修补：编写一个函数，通过补全缺失的引号、括号、花括号，尝试将残缺字符串转为合法 JSON。
- 状态机解析：逐字符扫描字符串，记录当前的嵌套深度和是否在字符串内，手动提取已完成的键值对。
- 库选型：使用类似 `partial-json-parser` 的现成方案，但需了解其内部正则或 AST 处理逻辑。
- 业务降级：如果解析失败，则回退到全量接收后再解析，避免 UI 报错。

常见坑：

- 错误地处理了转义字符（如 `\"`）。
- 在处理大型嵌套对象时，频繁的字符串修补操作导致性能瓶颈。

追问：

- 如果 AI 输出的是 Markdown 嵌套 JSON，你的解析策略会有什么变化？

8. Web Worker 在 AI 前端应用中有哪些典型的使用场景？

难度：★★

考点：多线程、性能优化

答案要点：

Web Worker 可以将耗时的计算任务从主线程剥离，确保 AI 聊天界面的交互（如滚动、输入）不卡顿。

- 大文本处理：对长达数万字的上下文进行分词（Tokenization）统计或向量化前的预处理。
- 语法高亮渲染：使用 Shiki 或 Prism 对流式输出的大段代码块进行高亮计算。
- 本地模型运行：如果使用 WebLLM 或 Transformers.js 在浏览器端跑模型，必须在 Worker 中运行。
- 数据持久化：在后台处理 IndexedDB 的大量对话记录写入和索引建立。

常见坑：

- 频繁在主线程和 Worker 间传递大数据量导致的序列化开销。
- 忘记处理 Worker 内部的错误捕获，导致后台任务静默失败。

追问：

- 如何在 Worker 中优雅地调用主线程定义的 API（如 Fetch）？

三面（架构与综合能力）

1. 如果让你从零设计一个支持 RAG（检索增强生成）的前端交互流程，你会考虑哪些关键点？

难度：★★★

考点：系统设计、AI 业务理解

答案要点：

RAG 流程比普通对话复杂，前端需要承担更多关于“引用来源”和“中间过程”的展示。

- 引用溯源 (Citations)：AI 回复中的每一个观点需要链接到原始文档，前端要实现点击跳转、悬浮预览原片断的功能。
- 检索过程可视化：展示“正在搜索知识库...”、“找到 3 篇相关文档”等中间状态，增强透明度。
- 反馈循环：允许用户对检索到的文档相关性进行打分（赞/踩），用于后端微调 Embedding 或重排算法。
- 混合搜索 UI：设计一个既能输入自然语言，又能筛选特定文件夹或文档范围的复合搜索框。

常见坑：

- 引用链接在流式输出过程中频繁跳动，导致用户无法点击。
- 忽略了文档权限控制，在前端展示了用户无权查看的敏感片段。

2. 在大模型应用中，前端如何参与 Token 成本控制和优化？

难度：★★★

考点：成本意识、技术方案

答案要点：

前端虽然不直接支付账单，但可以通过合理的策略显著减少不必要的 Token 消耗。

- 上下文裁剪策略：在发送请求前，前端根据模型窗口大小，智能裁剪历史消息（如保留最近 N 条或使用摘要）。
- 客户端缓存：利用 IndexedDB 缓存已生成的回复，对于完全相同的 Prompt（如常见的欢迎语或固定指令）直接命中缓存。
- 输入预校验：在前端计算 Token 数量（如使用 gpt-tokenizer），超过限制时提醒用户，避免无效请求。
- 任务拆解：将复杂任务拆分为多个小步骤，按需调用不同成本的模型（如简单分类用轻量模型，复杂推理用旗舰模型）。

常见坑：

- 裁剪历史记录时破坏了对话的连贯性。
- 客户端 Token 计算逻辑与后端不一致，导致误差。

3. 聊聊你在项目中遇到的最难的一个 AI 相关前端问题，你是如何解决的？

难度：★★★

考点：问题解决能力、技术深度

答案要点：

这是一个开放性问题，重点考察候选人在面对无先例问题时的思考路径。

- 问题背景：描述一个具体的场景（如：流式输出下的复杂图表渲染同步、多模态输入下的内存泄漏、移动端长对话的极致性能优化）。
- 调研过程：查阅了哪些文档（MDN, React 源码, 模型协议说明），做了哪些实验（对比测试, Profile 分析）。
- 方案对比：当时考虑了哪几种方案，为什么最终选择了这一种（权衡了开发成本、用户体验、扩展性）。
- 最终结果：上线后的数据表现（如卡顿率降低、用户留存提升）或团队内部的沉淀（如封装了通用组件库）。

常见坑：

- 描述的问题过于琐碎（如只是改了一个 CSS Bug）。
- 只有结论没有过程，无法体现技术深度。

4. 你认为 AI 时代的前端工程师，核心竞争力会发生怎样的转变？

难度：★★

考点：行业视野、职业规划

答案要点：

AI 正在改变前端的生產方式，核心竞争力将从“手写代码”转向“系统编排”和“交互定义”。

- 交互定义能力：从传统的组件交互转向自然语言交互（LUI）设计，关注如何引导用户与 AI 高效沟通。
- 模型应用能力：深入理解 LLM 的边界（幻觉、时效性），能够熟练运用 Prompt Engineering 和 Agent 架构解决业务问题。
- 全栈化趋势：前端需要更了解后端逻辑（如向量数据库、模型微调、边缘计算），以便设计更优的端到端方案。
- 提效工具利用：熟练使用 AI 辅助编程工具（Copilot, Cursor），将精力集中在架构设计和复杂逻辑攻克上。

常见坑：

- 表现出对 AI 的过度焦虑或完全排斥。
- 观点过于空洞，缺乏具体的实践支撑。

5. 如何评估一个 AI 交互功能的好坏？你会关注哪些前端指标？

难度：★★

考点：数据驱动、产品思维

答案要点：

除了传统的性能指标，AI 应用有一套独特的评价体系。

- 性能指标：TTFT（首个 Token 到达时间）、TPS（每秒生成 Token 数）、流式输出的稳定性。
- 交互指标：Prompt 的平均长度（反映用户表达成本）、对话轮数（反映任务完成效率）、重新生成率（反映结果满意度）。
- 质量指标：用户对 AI 回复的点赞/点踩率、引用链接的点击率、手动修改 AI 生成内容的比例。
- 异常指标：请求超时率、流中断率、非法 JSON 解析失败率。

常见坑：

- 只关注技术指标，忽略了用户最终是否解决了问题。
- 缺乏对比实验（A/B Test）意识。

6. 假设要实现一个类似 Canvas 的 AI 协作画布（如 Artifacts），你会选择什么样的技术栈，难点在哪？

难度：★★★

考点：复杂应用架构、技术选型

答案要点：

这种应用属于重前端逻辑的场景，需要处理实时通信、文档树管理和多端同步。

- 技术选型：使用 Yjs 或 Automerge 处理 CRDT 协同逻辑；渲染层可选 React + SVG/Canvas，或者集成现成的编辑器框架（如 Tldraw）。
- 通信机制：WebSocket 承载协同信令，SSE 承载 AI 生成的 DSL（如 React 代码或 SVG 指令）。
- 难点 1：AI 生成内容与用户手动修改内容的冲突合并（Merge 策略）。
- 难点 2：沙箱环境隔离，如何在前端安全地预览 AI 生成的第三方组件或代码。
- 难点 3：性能，在大规模画布节点下保持 60fps 的缩放和拖拽体验。

常见坑：

- 方案过于简单，没考虑到多人同时编辑时的状态一致性。
- 忽略了撤销/重做（Undo/Redo）在 AI 介入场景下的复杂性。